

"DBSCAN Clustering"

Problems with k-means

- ① Specify number of cluster before start. Although Elbow method curve can be used to find the number of clusters but sometimes it not work (invalid curve shape)
- ② As it is a distance based algorithm it is very sensitive to the outliers.
- ③ If data is distributed in complex shape it won't work. Only works for Spherical shape data.

To solve these problems DBSCAN comes.

#Density Based Clustering: cluster the data based on their density.

#Density Based Spatial Clustering of Applications with Noise.

→ an unsupervised algorithm used for clustering.

→ Density based ⇒ Clusters are defined by regions of high data density, not by distance to the centroid.

→ Non-parametric ⇒ It doesn't require to pre-define the number of cluster.

Why DBSCAN?

- ① Handles arbitrary shape
- ② No need for k
- ③ Robust to noise and outliers
- ④ Scales to large datasets.

When to use DBSCAN?

- ① contains clusters of varying shapes and size.
- ② number of clusters are unknown
- ③ dataset has noise and outliers.
- ④ clusters are not separated well.

DBSCAN revolves around 3 key points:

① Core points

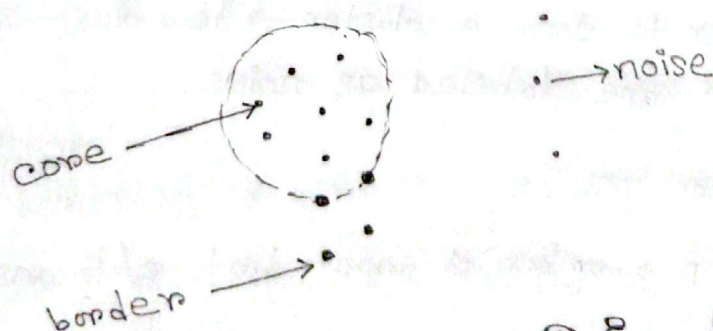
- a core point is one that sits in a dense region.
- To qualify as a core point, it must have enough neighbors (at least minPts) within a certain radius (ϵ).

② Border points

- a border point is close to a dense region (within ϵ distance of a core point) but not dense enough itself to be a core.
- border points belong to a cluster because they are near the core points but they cannot expand the cluster further.

③ Noise points:

- Too far from any dense region
- It doesn't have enough neighbors to be a core and it is not close enough to any core to be a border point.
- These points are ignored in clustering but are useful for anomaly detection task.



DBSCAN uses 2 parameters: ① ϵ ② minPts

Epsilon

ϵ is the neighborhood radius - the maximum distance that counts two points are neighbor.

It's how far we are willing to reach out from a point to see who's nearby.

Effect

Small $\epsilon \rightarrow$ very small neighborhoods $\rightarrow$ fewer neighbors per point $\rightarrow$ more points staged as noise or more tiny clusters.

Large $\epsilon \rightarrow$ big neighborhoods $\rightarrow$ points link across larger distances clusters grow in size and may merge with others.

Note ϵ is measured in the data's distance units, so feature scaling matters.

MinPts the minimum number of points required within a point's ϵ -neighborhood for that point to be considered as core point.

* MinPts includes itself also.

Intuition: How many neighbors are enough to call a location "dense."

Effect

small MinPts $\rightarrow$ easier to form clusters $\rightarrow$ more clusters, fewer noise points.

Large MinPts $\rightarrow$ harder to form a cluster $\rightarrow$ few clusters, more points are labeled as noise.

* Common starting rule for MinPts

- | | |
|----------------|--|
| ① $D + 1$ | $D \rightarrow$ number of input feature/dimensions |
| ② 4 to $2 * D$ | |

How ϵ and MinPts interact:

$\rightarrow$ They are not independent.

$\rightarrow$ If we shrink ϵ , we may also need to reduce MinPts to still find clusters.

$\rightarrow$ If we grow ϵ , we need to increase MinPts.

Trade-off of ϵ and MinPts

Tight ϵ + High MinPts $\rightarrow$ Strict density requirement $\rightarrow$ small number of confident clusters, lots of noise.

Loose ϵ + Low MinPts $\rightarrow$ permissive clustering $\rightarrow$ many/merged clusters, few noise.

Density Connected point:

A point A is density reachable from B if we can travel from B to A by hopping through a chain of core points.

* Density reachable is one way $\rightarrow$ If A is density reachable from B, the reverse may not be true unless both are cores.

Two point A and B are density connected if there exists a third point C such that:

- $\rightarrow$ A is density reachable from C and
- $\rightarrow$ B is " " " C.

If two points are density connected they belong to the same cluster.

Algorithm

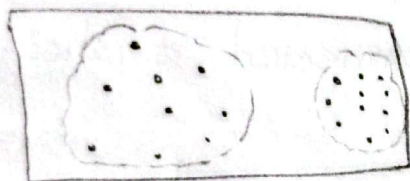
- ① Identity all points as core/border/noise point.
- ② For all the unclustered core points
 - Create a new cluster
 - add all the points that are unclustered and density connected to the current point into this cluster.
- ③ For each unclustered borders point assign it to the cluster of nearest core points.
- ④ Leave all the noise point as it is.

Pros of DBSCAN:

- ① No need to specify number of clusters, automatically find the cluster number.
- ② Find cluster of arbitrary shape.
- ③ Robust to outliers and noise.
- ④ Scales to large dataset, with the help of KDTree or Ball-Tree, it works efficiently on large data.
- ⑤ Deterministic: Results are stable.

Cons of DBSCAN:

- ① Highly sensitive to the parameter.
- ② Struggles with varying densities.



- ③ Border points ambiguity: 1 border point can belong to more than one cluster.
- ④ At high dimension distance metrics are less meaningful so there is a curse of dimensionality.
- ⑤ Not suitable for clusters with extreme size difference.

Why DBSCAN if it doesn't have Predict()?

- ① EDA → It is fantastic for discovering natural groupings in data where labels are missing.
- ② DBSCAN marks the outliers as -1 which is extremely useful in domain like fraud detection, identifying defective product.
- ③ To detect the actual shape of the clusters.

K-Distance graph:

- For each point in the dataset compute the distance to its k -nearest neighbors where $k = \text{MinPts}$.
 - Collect these distance for all points.
 - Sort the distances in descending order.
 - Plot them $\rightarrow$ distance on y-axis and points index on x-axis.
- This is called K-Distance graph.

This graph shows a sharp bend (elbow).

- Before the bend: points inside dense clusters $\rightarrow$ small distances to their k -th neighbor.
- After bend: points in sparse area or noise $\rightarrow$ large distances.

$\rightarrow$ MinPts are fixed (4 to $D+1$)

$\rightarrow$ K-distance helps to set value of ϵ

- Dense regions for clusters
- Sparse points beyond ϵ become noise.

#Silhouette Score:

• a metric used to evaluate the quality of clustering results. It measures how well each data point fits within its assigned cluster compared to other clusters.

It combines 2 concept:

① Cohesion (a): How close a point is to other points in the same cluster.

② Separation (b): How far a point is from points in the nearest different cluster.

for each point i :

$$s(i) = \frac{b(i) - a(i)}{\max(b(i), a(i))}$$

$a(i) \rightarrow$ avg distance from all other points in the same cluster.
 $b(i) \rightarrow$ avg distance of point i to all points in the nearest different cluster.